GUIDE

# Build a Personal Website with Cursor + GitHub Pages

A step-by-step guide to creating a personal website from scratch using Cursor and deploying it for free with GitHub Pages. No prior web development experience required.

---

## The Cursor-First Workflow

Before writing any code yourself, a static site can be built entirely through conversation with Cursor:

1. **Start from a prompt and an empty folder**
   - Open a new workspace in Cursor (for example, `my-site/` ).
   - Prompt Cursor to create a single-page website with:
     - `index.html` for structure and content.
     - `styles.css` for custom styling (optionally using a utility framework like Tailwind via CDN).
     - `script.js` for interactivity — scroll animations, mobile nav, etc.
   - Use follow-up prompts to shape the design: hero section, feature highlights, testimonials, color palette, dark/light theme.
2. **Iteratively refine content and layout**
   - Edit `index.html` via natural-language instructions:
     - Add, remove, or reorder sections (About, Projects, Experience, Contact, FAQ, etc.).
     - Refine your headline, value proposition, and calls-to-action.
   - Adjust `styles.css` :
     - Dial in your visual theme (colors, typography, spacing, background effects).
     - Make it responsive (mobile-first media queries, safe-area insets, flex-wrap).
     - Add accessibility basics ( `focus-visible` outlines, `prefers-reduced-motion` ).
   - Update `script.js` :
     - Add or remove interactive components (scroll reveals, counters, carousels, accordions).
     - Gate animations behind `prefers-reduced-motion` for accessibility.
     - Handle mobile menu toggling and active nav-link highlighting.
3. **Preview locally with Cursor's browser**

   - Run `python3 -m http.server 8765` from the project folder.
   - Use Cursor's built-in browser or a real device on the same Wi-Fi to check layout, spacing, and tap targets.
4. **(Optional) Keep a markdown content file**
   - Maintain a file like `CONTENT.md` as a single source of truth for all copy — headlines, bios, project descriptions, testimonials.
   - Periodically sync `index.html` to match it (and vice versa).
5. **Create a Git repo and push to GitHub**
   - Initialize git, commit site files, create a GitHub repo, and push `main` .
   - Make sure the site files live at the repo root so GitHub Pages can serve them directly.
6. **Enable GitHub Pages**
   - Turn on Pages in repo settings (source: `main` , folder: `/` ).
   - From this point the workflow is: **edit → preview → commit → push → live in ~2 minutes**.

## 1. Prerequisites

- **Accounts**
  - A GitHub account with SSH or HTTPS access configured.
- **Local tools**
  - `git` — verify with `git --version`
  - Python 3 — verify with `python3 --version` (used for a simple local server)
  - Cursor IDE installed

## 2. Create (or clone) the repository

### Option A: Start from scratch

```
mkdir my-site && cd my-site
git init
```

Open the folder in Cursor and start prompting to generate your site files.

### Option B: Clone an existing repo

```
git clone git@github.com:<your-username>/<your-repo>.git
cd <your-repo>
```

A typical project structure looks like:

```
<your-repo>/
├── index.html
├── styles.css
├── script.js
├── favicon.svg
├── images/
│   └── (your images here)
└── README.md
```

## 3. Open the project in Cursor

1. Open Cursor.
2. **File → Open Folder...** and select your project folder.
3. Cursor will index the files. You can now use Agent mode or inline edits to modify any file with natural language.

## 4. Run a local preview server

From a terminal inside the project folder:

```
python3 -m http.server 8765
```

- **Desktop**: open `http://localhost:8765/` in your browser.
- **Mobile** (same Wi-Fi): find your computer's local IP (e.g. `192.168.1.42` ) and open `http://192.168.1.42:8765/` on your phone.

Stop the server with `Ctrl+C` when finished.

## 5. Editing the site in Cursor

### 5.1 HTML ( `index.html` )

- **Hero & headline**: Your name/title, a one-liner about what you do, and a primary CTA.
- **Core sections**: About, Projects/Work, Skills, Experience, Contact — whatever fits your story.
- **Social proof**: Testimonials, client logos, ratings, or notable achievements.

If your template uses a wrapper class (like `.container` or `.site-container` ), keep new sections inside it for consistent alignment.

### 5.2 CSS ( `styles.css` )

- Theme variables (colors, fonts) are usually defined at the top in `:root` .

- A wrapper class controls max-width and horizontal padding across sections.
- Media queries handle mobile layout, stacking, and safe-area padding.

Make small changes, verify in the browser, then commit.

### 5.3 JavaScript ( `script.js` )

- Common behaviors for personal sites:
  - Scroll-reveal animations via `IntersectionObserver` .
  - Stat or number counter animations (gated behind `prefers-reduced-motion` ).
  - Mobile hamburger menu toggle.
  - Active nav-link highlighting on scroll.

Only touch JS when you want to change or add interactive behavior.

## 6. Commit your changes

After editing and verifying locally:

```
git status

git add index.html styles.css script.js
# (add any other changed files)

git commit -m "Update hero section and add projects"
```

To unstage a file you didn't mean to include:

```
git restore --staged <file>
```

## 7. Push to GitHub

```
git push
```

If this is your first push, Git may ask you to authenticate via browser or a Personal Access Token. After that, subsequent pushes are automatic.

## 8. Configure GitHub Pages (one-time setup)

1. Go to your repo on GitHub: `https://github.com/<your-username>/<your-repo>`
2. Click **Settings** → **Pages** (in the left sidebar).

3. Under **Source**, select:
   - `Deploy from a branch`
   - Branch: `main`
   - Folder: `/ (root)`
4. Click **Save**.

After 1–3 minutes your site will be live at:

```
https://<your-username>.github.io/<your-repo>/
```

The Pages settings page shows the exact URL and deployment status.

## 9. Updating the live site (quick path)

For small, low-risk changes you can push directly to `main` :

1. **Edit** in Cursor (HTML / CSS / JS).
2. **Preview** at `http://localhost:8765/` .
3. **Commit and push**:

   ```
   git add .
   git commit -m "Short description of change"
   git push
   ```

4. Wait ~1–3 minutes for GitHub Pages to rebuild.
5. Visit your live URL. If it looks cached, use a private/incognito tab or append `?v=2` to the URL.

## 10. Recommended: Feature Branch + Pull Request Workflow

For any meaningful update, use a **feature branch** and **pull request** instead of pushing directly to `main` . This gives you a chance to review the change in GitHub before it goes live — catching mistakes, broken layouts, or copy errors before they reach your audience.

### Why this matters

- **Protects your live site.** Changes on a feature branch don't affect `main` , so your published site stays stable while you iterate.
- **Enables code review.** The pull request diff shows exactly what changed, making it easy to spot typos, missing files, or unintended edits before merging.
- **Creates a history.** Each PR is a self-contained record of what changed and why, which is valuable if you ever need to revert or understand past decisions.

### The steps

1. **Create a feature branch** before making any edits:

```
git checkout -b feature/my-change
```

2. **Tell Cursor what to change** — describe the edit in plain English (e.g. "Update the hero headline to X" or "Add a Projects section with three cards"). Cursor modifies the code for you.
3. **Preview locally** — check the result at `http://localhost:8765/` on desktop and mobile.
4. **Commit and push** to the feature branch (not `main`):

```
git add .
git commit -m "feat: describe the change"
git push -u origin feature/my-change
```

5. **Open a pull request** from `feature/my-change` → `main`. You can do this on GitHub or with the GitHub CLI:

```
gh pr create --title "feat: describe the change" --body "Summary of wh
```

6. **Review the PR** on GitHub — check the diff, verify the changes look correct.
7. **Merge the PR** — once you're satisfied, merge it into `main`. GitHub Pages will automatically rebuild and your site goes live in ~1–3 minutes.

### Using Cursor shortcut commands

If your project has Cursor skills configured, the workflow simplifies to:

1. Create a feature branch: `git checkout -b feature/my-change`
2. Tell Cursor what to change
3. Preview at `http://localhost:8765/`
4. **`/commit`** — stages, commits, and pushes to the feature branch
5. **`/pr`** — opens a pull request from the feature branch → `main`
6. Review and merge the PR on GitHub — site deploys automatically

## 11. Optional: Use as your root GitHub Pages site

By default, project sites live at `https://<your-username>.github.io/<your-repo>/`. If you want the shorter root URL:

```
https://<your-username>.github.io
```

You need a repo named exactly `<your-username>.github.io` :

1. Create a repo called `<your-username>.github.io` .
2. Copy your site files into that repo's root.
3. Enable Pages (branch: `main` , folder: `/` ).

Your site will then be served at the root domain, and your original project repo can remain as a separate project site.

## 12. Workflow summary

1. Create or clone a repo and open it in Cursor.
2. Run `python3 -m http.server 8765` to preview locally.
3. Use Cursor to edit `index.html` , `styles.css` , and `script.js` with natural language.
4. Verify changes in the browser (desktop + mobile).
5. `git add . && git commit -m "message" && git push`
6. Wait ~2 minutes for GitHub Pages to redeploy.
7. Visit `https://<your-username>.github.io/<your-repo>/` and confirm.

---

© 2026 Jon Metz